of functions rather than state and mutable data, as opposed to Imperative Programming. In other words, the emphasis is on functions and their evaluation, rather than achieving results by directing the program through a variety of states, which is achieved in different methods. In simpler terms, variables, data structures, I/O, and assignments whose values at each execution determine the state (which leads to reliability issues) are avoided.

In pure functional languages, I/O, assignments, etc, are completely avoided. In Python, however, we do not avoid them completely. Instead, a functional-like interface is provided, but internally, non-functional features are used. For example, local variables are used inside the function, but global variables outside the function will not be modified.

Functional programming can be regarded as the opposite of object-oriented programming. Objects are entities that provide functions to modify internal variables (and hence internal state). Functional programming aims at the complete elimination of state (as much as possible), and works with the data flow between functions.

In Python, it is possible to combine object-oriented and functional programming. For example, functions could receive and return instances of objects.

The advantages of functional programming

A main feature of functional programming is the use of pure functions (functions that return the same output every time, for the same input). A simple example is the function *sinx*, which returns the same output for the same input, as opposed to the function *today*, which returns different values depending on when it is executed.

Another feature is that functions are first-class objects, i.e., they can be created dynamically and can be passed as arguments to, and even returned as values from, functions.

A key feature of functional programming is referential integrity, i.e., the lack of side effects. The execution of functions does not affect state, and hence eliminates deviation from the intended behaviour. This means that it is much easier to verify, parallelise and optimise programs, as well as write automated tools to perform the same tasks.

There are several theoretical and practical advantages of functional programming, which are beyond the scope of this article. Readers are encouraged to refer to the links provided.

Functional programming in Python

In this section, I will cover a few constructs that enable writing functional style programs in Python. All code in this section was run on Python 2.6.6 over GCC 4.4.5 in Ubuntu 10.10.

Iterators

Let's start by looking at an important foundation for writing functional-style programs in Python: iterators. These are objects that represent a stream of data, and return one data element at a time. These are extremely useful in manipulating lists, tuples, strings, etc. Python has a built-in function *iter()*, which takes an object and returns an iterator to it. Consider the following code, which creates an iterator object, *it*, for the list *L*. Notice how each element is accessed using the *next()* method of the iterator object.

```
>>> L = [1,2,3]
>>> it = iter(L)
>>> print it
<listiterator object at 0x7f1ec1f9f550>
>>> it.next()
1
>>> it.next()
2
>>> it.next()
3
>>> it.next()
Traceback (most recent call last):
File "<stdin>", line 1, in ?
StopIteration
>>>
```

Note: If *iter()* raises a *TypeError*, it means that the object passed to it doesn't support iteration.

Iterators are most commonly used in *for* statements in Python, as shown below:

```
for i in iter(alist):
     print i
```

Another way of achieving the same is as follows:

```
for i in alist:
     print i
```

Calling *iter()* on a dictionary returns an iterator that loops over the keys:

```
>>> monthList={'Jan' : 1, 'Feb' : 2, 'Mar' : 3, 'Apr' : 4,
'May' : 5, 'Jun' : 6, 'Jul' : 7, 'Aug' : 8, 'Sep' : 9, 'Oct' :
10, 'Nov' : 11, 'Dec' : 12}
>>> for key in monthList:
```